COAL LAB – FALL 2026
LAB #06 – 23K-0060

TASK – 1:

Code;

```
INCLUDE Irvine32.inc

.data
marks BYTE 67, 84, 92, 75, 88

msg1 BYTE "First element: ", 0
msg2 BYTE "Last element: ", 0
msg3 BYTE "Element at index 2: ", 0
msg4 BYTE "Updated array: ", 0
msg5 BYTE "Number of elements: ", 0
msg6 BYTE "Total size in bytes: ", 0

.code
main PROC

    ; Display first element
    mov edx, OFFSET msg1
    call WriteString
    movzx eax, marks[0]
    call WriteDec
    call Crlf

    ; Display last element
    mov edx, OFFSET msg2
    call WriteString
    movzx eax, marks[LENGTHOF marks - 1]
    call WriteDec
    call Crlf

    ; Display element at index 2
    mov edx, OFFSET msg3
```

```
    call WriteString
    movzx eax, marks[2]
    call WriteDec
    call Crlf

    ; Update second element to 90
    mov marks[1], 90

    ; Display updated array
    mov edx, OFFSET msg4
    call WriteString

    mov esi, OFFSET marks
    mov ecx, LENGTHOF marks

L1:
    movzx eax, BYTE PTR [esi]
    call WriteDec
    mov al, ' '
    call WriteChar
    add esi, TYPE marks
    loop L1

    call Crlf

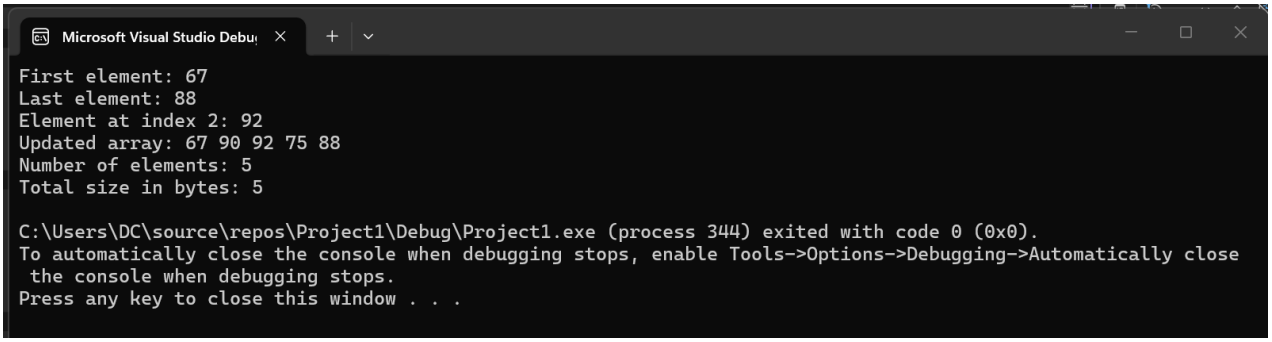
    ; Display array length
    mov edx, OFFSET msg5
    call WriteString
    mov eax, LENGTHOF marks
    call WriteDec
    call Crlf

    ; Display total size
    mov edx, OFFSET msg6
    call WriteString
    mov eax, SIZEOF marks
    call WriteDec
    call Crlf

    exit

main ENDP
END main
```

Output;



TASK – 2:
Code;

```
INCLUDE Irvine32.inc

.data
scores DWORD 12, 25, 18, 30, 15, 20

msg1 BYTE "Array elements: ", 0
msg2 BYTE "Total: ", 0
msg3 BYTE "Average: ", 0

total DWORD 0

.code
main PROC

    mov edx, OFFSET msg1
    call WriteString

    mov esi, 0
    mov ecx, LENGTHOF scores
    mov ebx, 0

L1:
    mov eax, scores[esi * TYPE scores]
    call WriteDec

    add ebx, eax

    mov al, ' '
```

```
    call WriteChar

    inc esi
    loop L1

    call Crlf

    ; Save total
    mov total, ebx

    ; Display total
    mov edx, OFFSET msg2
    call WriteString
    mov eax, total
    call WriteDec
    call Crlf

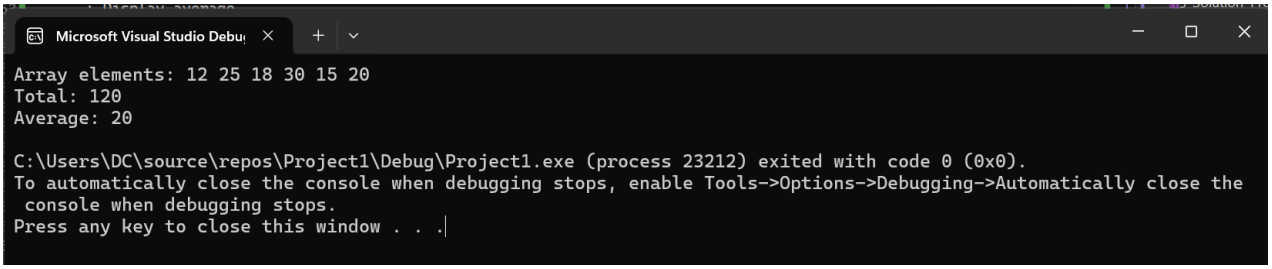
    ; Calculate integer average
    mov eax, total
    mov edx, 0
    mov ebx, LENGTHOF scores
    div ebx

    ; Display average
    mov ebx, eax
    mov edx, OFFSET msg3
    call WriteString
    mov eax, ebx
    call WriteDec
    call Crlf

    exit

main ENDP
END main
```

Output;



TASK – 3:
Code;

```
INCLUDE Irvine32.inc

.data
multiples DWORD 10 DUP(?)

msg1 BYTE "First ten multiples of 7: ", 0
msg2 BYTE "Memory dump:", 0

.code
main PROC

    ; Initialize pointer and loop count
    mov esi, OFFSET multiples
    mov ecx, LENGTHOF multiples
    mov eax, 7

L1:
    ; Store current multiple
    mov [esi], eax

    ; Generate next multiple
    add eax, 7

    ; Move pointer to next DWORD
    add esi, TYPE multiples

    loop L1

    ; Display array
    mov edx, OFFSET msg1
    call WriteString

    mov esi, OFFSET multiples
    mov ecx, LENGTHOF multiples

L2:
    mov eax, [esi]
    call WriteDec

    mov al, ' '
    call WriteChar
```

```
    add esi, TYPE multiples
    loop L2

    call Crlf

    ; Display memory contents
    mov edx, OFFSET msg2
    call WriteString
    call Crlf

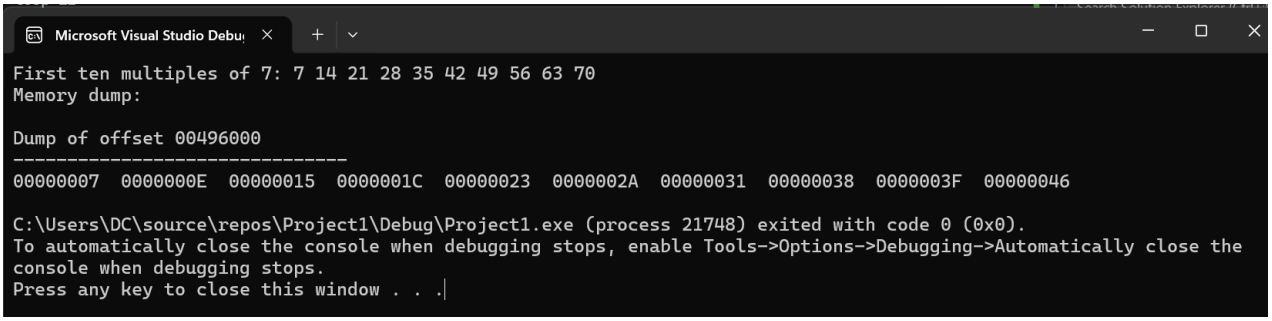
    mov esi, OFFSET multiples
    mov ecx, LENGTHOF multiples
    mov ebx, TYPE multiples

    call DumpMem

    exit

main ENDP
END main
```

Output;



TASK – 4:

Code;

```
INCLUDE Irvine32.inc

.data
outerCount DWORD ?
rowNum DWORD 1
num DWORD ?
```

```
.code
main PROC

    ; Outer loop controls five rows
    mov ecx, 5

OuterLoop:
    ; Save outer loop counter
    mov outerCount, ecx

    ; Set number of values in this row
    mov ecx, rowNum
    mov num, 1

InnerLoop:
    ; Display current number
    mov eax, num
    call WriteDec

    ; Print space
    mov al, ' '
    call WriteChar

    ; Increase number
    inc num

    loop InnerLoop

    ; Move to next row
    call Crlf

    ; Increase row length
    inc rowNum

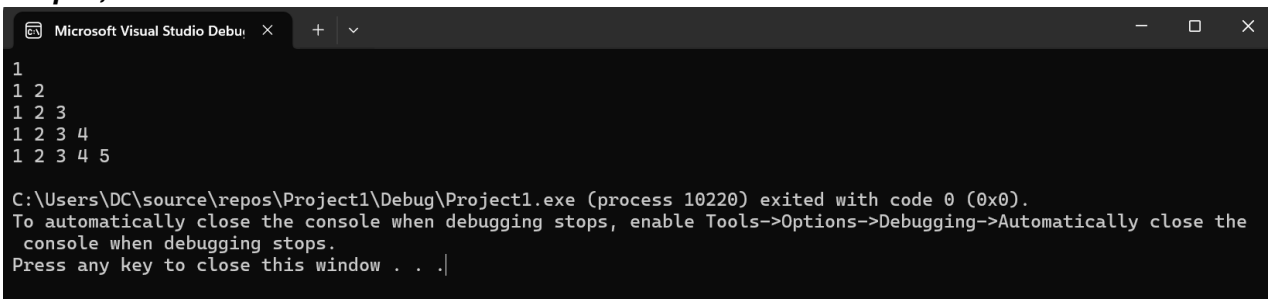
    ; Restore outer loop counter
    mov ecx, outerCount

    loop OuterLoop

    exit

main ENDP
END main
```

Output;



TASK – 5:
Code;

```
INCLUDE Irvine32.inc

.data
rolls DWORD 12 DUP(?)

msg1 BYTE "Dice rolls: ", 0
msg2 BYTE "Count of rolls >= 4: ", 0
msg3 BYTE "Sum of all rolls: ", 0
msg4 BYTE "Results cancelled by user.", 0

question BYTE "Do you want to view the results?", 0
caption BYTE "Dice Analysis", 0

countHigh DWORD 0
total DWORD 0

.code
main PROC

    ; Initialize random number generator
    call Randomize

    ; Generate 12 dice rolls
    mov esi, OFFSET rolls
    mov ecx, LENGTHOF rolls

GenerateLoop:
    mov eax, 6
```

```
    call RandomRange

    ; Convert 0-5 into 1-6
    inc eax

    mov [esi], eax
    add esi, TYPE rolls

    loop GenerateLoop

    ; Display dice rolls
    mov edx, OFFSET msg1
    call WriteString

    mov esi, OFFSET rolls
    mov ecx, LENGTHOF rolls

DisplayLoop:
    mov eax, [esi]
    call WriteDec

    mov al, ' '
    call WriteChar

    add esi, TYPE rolls
    loop DisplayLoop

    call Crlf

    ; Calculate sum and count rolls >= 4
    mov esi, OFFSET rolls
    mov ecx, LENGTHOF rolls

AnalyzeLoop:
    mov eax, [esi]

    ; Add roll to total
    add total, eax

    ; Check whether roll >= 4
    cmp eax, 4
    jl SkipCount
```

```
    inc countHigh

SkipCount:
    add esi, TYPE rolls
    loop AnalyzeLoop

    ; Ask user to view results
    mov edx, OFFSET question
    mov ebx, OFFSET caption
    call MsgBoxAsk

    ; EAX = 6 means Yes
    cmp eax, 6
    je ShowResults

    ; EAX = 7 means No
    cmp eax, 7
    je CancelResults

    ; Handle any other response as cancellation
    jmp CancelResults

ShowResults:
    mov edx, OFFSET msg2
    call WriteString

    mov eax, countHigh
    call WriteDec
    call Crlf

    mov edx, OFFSET msg3
    call WriteString

    mov eax, total
    call WriteDec
    call Crlf

    jmp ProgramEnd

CancelResults:
    mov edx, OFFSET msg4
    call WriteString
    call Crlf
```

```
ProgramEnd:
    exit

main ENDP
END main
```

Output;

